Course Grading Rubric

For Programming Assignments and Projects

ERIC REES, PHD
TEXAS TECH UNIVERSITY
Department of Computer Science

Course Materials ©2025 by Eric Rees is licensed under CC BY-NC-ND 4.0

Contents

1. Rubric Criteria 2
1.1 Program Specifications & Correctness 2
1.2 Readability 3
1.3 Documentation 4
1.4 Code Efficiency 5
1.5 Assignment Specifications 5
2. Example File Headers 6
Bash 6
C 6
C++ 6
C# 7
Java 7
MASM Assembly 7
Prolog 8
Python 8
Scheme 8
3. Grading Rubric Table 9

The purpose of this document is to lay out the common criteria used to grade CS 3361 programming assignments. Each criterion listed has four different levels of mastery, with a description of how a submission will attain each level and the number of points awarded for achieving it.

1. Rubric Criteria

1.1 Program Specifications & Correctness

This criterion is about how well the program meets the given specifications or requirements and how correctly it performs its intended functions. For example, if the assignment is to write a program that sorts a list of numbers in ascending order, the program would be evaluated based on whether it indeed sorts the list correctly. An 'Excellent' program would sort all types and sizes of lists correctly without any errors. An 'Adequate' program might sort most lists correctly but may have minor issues with edge cases (like an empty list or a list with duplicate numbers). A 'Poor' program might only sort some lists correctly, and a program with 'Not Met' might not sort the list at all or not even run.

This is the most important criterion as any submitted program must function correctly and meet the specifications given. Submitted programs should always behave as desired and produce correct output and results for a variety of inputs. This criterion also includes the need to meet all specifications laid out in the problem statement by writing a program in a particular way, using a particular language feature, or not using a particular language feature. For example, if the specifications stated you must use a particular library then you may go to “Poor” or “Not Met” if you simply ignore this requirement and use a different library.

If you believe a specification to be ambiguous or unclear, you should consult with the instructor instead of making any assumptions.

Program Specifications and Correctness (60%)

- Excellent (60 points)
  - The program perfectly meets all specifications and requirements.
  - It correctly performs all functionalities and passes all test cases without any errors or exceptions.
- Good (45 points)
  - The program mostly meets all specifications and requirements.
  - There might be minor errors or issues, but the overall functionality is preserved.
  - Most (>=80) test cases are passed successfully.
- Limited (30 points)
  - The program partially meets the specifications and requirements.
  - There are significant errors that affect functionality.
  - It passes barely over half of the test cases.
- Not Met (0 points)
  - The program does not meet the specifications and requirements.
  - There are major errors or the program does not run at all.
  - It fails most or all test cases.

1.2 Readability

Readability is about how easy it is to understand the code. This includes the structure and organization of the code, the naming of variables and functions, and the use of whitespace and indentation. For instance, an 'Excellent' program might have a clear structure with each function performing a single task, variables named descriptively (like 'sorted_list' instead of 'sl'), and consistent use of indentation and whitespace for readability. An 'Adequate' program might have generally good readability but with occasional inconsistencies. A 'Poor' program would be difficult to understand due to poor naming, lack of structure, or inconsistent formatting. A 'Not Met' program would be almost unintelligible due to a complete lack of structure and organization.

Code should be readable to both you as well as a knowledgeable third party. This involves:

- Using indentation consistently.
- Adding whitespace (blank lines or spaces) where appropriate to help distinguish parts of the program.
    - Examples:
        - Space after commas in a list
        - Blank lines between functions or between blocks of related lines within function.
- Using meaningful variable names. Variables with names like ***A***, ***B***, ***C***, ***foo***, or ***bar*** give the reader no information regarding the variable's purpose or what information it may contain. Names like ***maximum***, ***counter*** or ***inputString*** are considerably more useful and make their purpose known. Loop variable names are an exception to this rule and names like ***x***, ***y***, ***i***, or ***j*** are allowed for loop variables.
- Properly organizing code to increase readability and reusability. Code should be organized into functions so that blocks of code that can be re-used are contained within functions that enable that behavior.

Readability (15%)

- Excellent (15 points)
    - The code is exceptionally well-structured and organized.
    - Naming conventions are consistently followed, making the code self-explanatory.
- Good (10 points)
    - The code is generally well-structured and organized.
    - There might be occasional inconsistencies in naming conventions or slight lack of organization.
- Limited (5 points)
    - The code lacks structure and organization.
    - Naming conventions are inconsistently followed, making it difficult to understand the code.
- Not Met (0 points)
    - The code is not structured or organized.
    - Naming conventions are not followed, making the code unintelligible.

1.3 Documentation

Documentation refers to the comments and explanations included in the code that help explain what the code does and why certain decisions were made. An 'Excellent' program would have comprehensive comments explaining the purpose and functionality of each function, any complex or tricky parts of the code, and the overall approach taken. An 'Adequate' program might have generally good comments, but some parts of the code may lack necessary explanation. A 'Poor' program would have few comments, leaving much of the code unexplained. A 'Not Met' program would have no comments or explanations at all.

Every file you submit that contains source code should start with consistent header comments that lay out the following information:

1) Title of the source code file and which package(s) it belongs to.
2) Description of the source code and what functions it performs.
3) Name of the author and their R#.
4) The date (either date of beginning, date of last update, or date of submission).
5) Version number.
6) Usage notes including how the program is utilized and if it has any required inputs.
7) General Notes giving any general information or stating in require prerequisites or requirements.
8) Any required versioning information (compiler or interpreter version).

2. Example File Headers gives you examples you should utilize for your submissions. Find the language your code is being written in and feel free to utilize that example.

All code in your program should also be well-commented. This requires that you strike a balance between:

1) Being overly verbose and commenting everything – which adds a great deal of unneeded noise to the code, and
2) Writing no comments in the code – which adds a great deal of unnecessary complexity to the code by not giving any assistance to future readers

Documentation (10%)

- Excellent (10 points)
  - The code is thoroughly documented.
  - All functions and complex code blocks have clear, concise comments explaining their purpose and functionality.
- Good (8 points)
  - The code is mostly documented.
  - Most functions and complex code blocks have comments, but some might lack clarity or completeness.
- Limited (5 points)
  - The code has minimal documentation or is completely missing the required file header.
  - Many functions and complex code blocks lack comments.
- Not Met (0 points)
  - The code has no documentation or comments.

1.4 Code Efficiency

There are many ways to write the same functionality into your code, and many of them may be poor choices. They may be poor choices because they require more lines of code than are necessary, they take considerably longer to execute than necessary, or they consume considerably more system resources (such as RAM) during execution than necessary. Whenever possible, code should be concise, stray from using overly complicated formulas, and use the most efficient algorithms for solving the problem at hand.

As such, code efficiency refers to how well the code makes use of resources like time and memory. An 'Excellent' program would use the most efficient algorithms and data structures for the task, resulting in a fast and memory-efficient program. An 'Adequate' program might generally be efficient but could have some parts of the code that could be optimized further. A 'Poor' program might work correctly but would be unnecessarily slow or use more memory than needed due to inefficient code. A 'Not Met' program would make no apparent attempt at optimization, resulting in a slow and resource-hungry program.

Code Efficiency (5%)

- Excellent (5 points)
    - The code is generally efficient.
    - Best practices for resource management and optimization are often followed. However, there might be occasional inefficiencies or missed opportunities for optimization.
- Limited (2 points)
    - The code shows limited efficiency.
    - There are significant areas where optimization and resource management could be improved.
- Not Met (0 points)
    - The code is not efficient.
    - There is no evidence of attempts to optimize or manage resources effectively.

1.5 Assignment Specifications

Programming assignments in this course will often contain specifications or requirements beyond those required to solve the problem itself. Assignment Specification is about following the specific requirements given for the assignment. This might include things like the file format to submit in, the naming of files, the inclusion of certain elements like a README file, or any other specific instructions given. An 'Excellent' program would follow all these specifications exactly. A 'Poor' program would miss several specifications, and a 'Not Met' program would not follow the specifications at all.

Assignment Specification (10%)

- Excellent (10 points)
    - All assignment specifications are met, including file format, submission guidelines, and any other specific requirements.
- Limited (5 points)
    - Some assignment specifications are met, but there are significant deviations from the requirements.
- Not Met (0 points)
    - The assignment specifications are not met or there are major deviations from the requirements.

2. Example File Headers

Bash

```
#!/bin/bash -
#=====================================================================
#title        : example.sh
#description  : This is an example script.
#author       : errees (R#123456)
#date         : 01/01/2001
#version      : 1.0
#usage        : bash example.sh
#notes        : This example script has no requirements.
#bash_version : 4.2.46(2)-release
#=====================================================================
```

C

```
/*
===========================================================================
Title        : example.c
Description  : This is an example program.
Author       : errees (R#123456)
Date         : 01/01/2001
Version      : 1.0
Usage        : Compile and run this program using the GNU C compiler
Notes        : This example program has no requirements.
C Version    : Specify your C version
===========================================================================
*/
```

C++

```
/*
===========================================================================
Title        : example.cpp
Description  : This is an example program.
Author       : errees (R#123456)
Date         : 01/01/2001
Version      : 1.0
Usage        : Compile and run this program using the GNU C++ compiler
Notes        : This example program has no requirements.
C++ Version  : Specify your C++ version
===========================================================================
*/
```

C#

```
/*
==============================================================================
Title        : Example.cs
Description  : This is an example program.
Author       : errees (R#123456)
Date         : 01/01/2001
Version      : 1.0
Usage        : Compile and run this program using your C# compiler
Notes        : This example program has no requirements.
C# Version   : Specify your C# version
==============================================================================
*/
```

Java

```
/*
==============================================================================
Title        : Example.java
Description  : This is an example program.
Author       : errees (R#123456)
Date         : 01/01/2001
Version      : 1.0
Usage        : Compile and run this program using your Java compiler
Notes        : This example program has no requirements.
Java Version : Specify your Java version
==============================================================================
*/
```

MASM Assembly

```
;==============================================================================
=
; Title        : example.asm
; Description  : This is an example script.
; Author       : errees (R#123456)
; Date         : 01/01/2001
; Version      : 1.0
; Usage        : Assemble and link using your MASM compiler
; Notes        : This example script has no requirements.
; MASM Version : Specify your MASM version
;==============================================================================
=
```

Prolog

```
/*
==============================================================================
Title        : example.pl
Description  : This is an example script.
Author       : errees (R#123456)
Date         : 01/01/2001
Version      : 1.0
Usage        : swipl -s example.pl
Notes        : This example script has no requirements.
Prolog Version: Specify your Prolog version
==============================================================================
*/
```

Python

```
#!/usr/bin/env python3
"""
==============================================================================
Title        : example.py
Description  : This is an example script.
Author       : errees (R#123456)
Date         : 01/01/2001
Version      : 1.0
Usage        : python3 example.py
Notes        : This example script has no requirements.
Python Version: 3.x.x
==============================================================================
"""
```

Scheme

```
;
;==============================================================================
; Title        : example.scm
; Description  : This is an example script.
; Author       : errees (R#123456)
; Date         : 01/01/2001
; Version      : 1.0
; Usage        : scheme --script example.scm
; Notes        : This example script has no requirements.
; Scheme Version: Specify your Scheme version
;==============================================================================
;
```

3. Grading Rubric Table

Criterion	Points	Excellent	Good	Limited	Not Met
*Program Specifications & Correctness**	60	**60 Points** No errors, program always works correctly and meets the specification(s).	**45 Points** Minor details of the program specification are violated, program functions incorrectly for some inputs.	**30 points** Significant details of the specification are violated and/or program often exhibits incorrect behavior.	**0 points** Program only functions correctly in very limited cases or not at all.
Readability	15	**15 points** No errors, code is clean, understandable, and well-organized.	**10 points** Minor issues with consistent indentation, use of whitespace, variable naming, or general organization.	**5 points** At least one major issue with indentation, whitespace, variable names, or organization.	**0 Points** Major problems with at three or four of the readability subcategories.
Documentation	10	**10 points** No errors, code is well-commented.	**8 points** One or two places that could benefit from comments are missing them or the code is overly commented.	**5 points** File header missing, complicated lines or sections of code uncommented or lacking meaningful comments.	**0 points** No file header or comments present.
Code Efficiency	5	**5 points** No errors, code uses the best approach in every case.		**2 points** Code uses poorly-chosen approaches in at least one place.	**0 points** Many things in the code could have been accomplished in an easier, faster, or otherwise better fashion.
Assignment Specifications	10	**10 points** No errors.		**5 points** Minor details of the assignment specification are violated, such as files named incorrectly or extra instructions slightly misunderstood.	**0 points** Significant details of the specification are violated, such as extra instructions ignored or entirely misunderstood.

** As a special case, if a program does not meet the specifications at all or is entirely incorrect, no credit will be received for the other criteria either.*